

AG3 - Actividad Guiada 3

Nombre: Raul Reyero

Link: https://colab.research.google.com/drive/1fM8_y8p-L2T5rG7xa5oY5hQCxcrAr3R4?usp=sharing

Github: <https://github.com/rubengp02/O3MIAR-Algoritmos-de-Optimizacion>

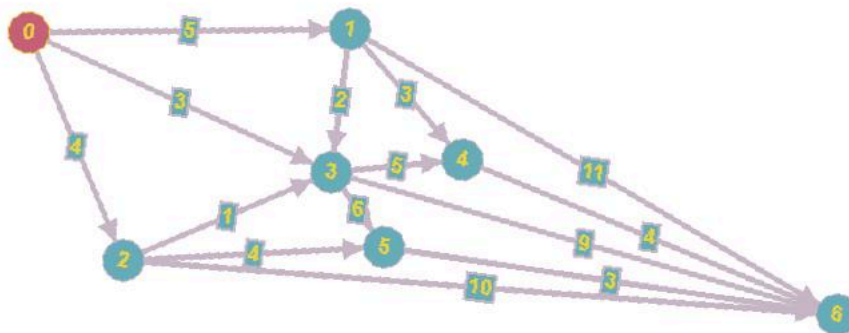
```
import math
```

Programación Dinámica. Viaje por el río

- **Definición:** Es posible dividir el problema en subproblemas más pequeños, guardando las soluciones para ser utilizadas más adelante.
- **Características** que permiten identificar problemas aplicables:
 - Es posible almacenar soluciones de los subproblemas para ser utilizados más adelante
 - Debe verificar el principio de optimalidad de Bellman: "en una secuencia optima de decisiones, toda sub-secuencia también es óptima" (*)
 - La necesidad de guardar la información acerca de las soluciones parciales unido a la recursividad provoca la necesidad de preocuparnos por la complejidad espacial (cuantos recursos de espacio usaremos)

Problema

En un río hay n embarcaderos y debemos desplazarnos río abajo desde un embarcadero a otro. Cada embarcadero tiene precios diferentes para ir de un embarcadero a otro situado más abajo. Para ir del embarcadero i al j , puede ocurrir que sea más barato hacer un trasbordo por un embarcadero intermedio k . El problema consiste en determinar la combinación más barata.



*Consideramos una tabla $TARIFAS(i,j)$ para almacenar todos los precios que nos ofrecen los embarcaderos.

*Si no es posible ir desde i a j daremos un valor alto para garantizar que ese trayecto no se va a elegir en la ruta óptima(modelado habitual para restricciones)

```
#Viaje por el río - Programación dinámica
#####

TARIFAS = [
[0,5,4,3,float("inf"),float("inf"),float("inf")], #desde nodo 0
[float("inf"),0,float("inf"),2,3,float("inf"),11], #desde nodo 1
[float("inf"),float("inf"), 0,1,float("inf"),4,10], #desde nodo 2
[float("inf"),float("inf"),float("inf"), 0,5,6,9],
[float("inf"),float("inf"), float("inf"),float("inf"),0,float("inf"),4],
[float("inf"),float("inf"), float("inf"),float("inf"),float("inf"),0,3],
[float("inf"),float("inf"),float("inf"),float("inf"),float("inf"),float("inf"),0]
]

# Se pone valor inf cuando no hay conexión directa entre nodos y cuando se va hacia atrás (solo se debe avanzar)
TARIFAS

[[0, 5, 4, 3, inf, inf, inf],
[inf, 0, inf, 2, 3, inf, 11],
[inf, inf, 0, 1, inf, 4, 10],
[inf, inf, inf, 0, 5, 6, 9],
[inf, inf, inf, inf, 0, inf, 4],
```

La ruta pasa por :

```
0,2,5
Con valor 11
```

▼ Problema de Asignacion de tarea

```
#Asignacion de tareas - Ramificación y Poda
#####
#   T A R E A
#   A
#   G
#   E
#   N
#   T
#   E

COSTES=[[11,12,18,40],
         [14,15,13,22],
         [11,17,19,23],
         [17,14,20,28]]
```

```
#Calculo del valor de una solucion parcial
def valor(S,COSTES):
    VALOR = 0
    for i in range(len(S)):
        VALOR += COSTES[S[i]][i]
    return VALOR
```

```
valor((0, 1, 2, 3 ),COSTES)
```

73

```
#Coste inferior para soluciones parciales
# (1,3,) Se asigna la tarea 1 al agente 0 y la tarea 3 al agente 1

def CI(S,COSTES):
    VALOR = 0
    #Valores establecidos
    for i in range(len(S)):
        VALOR += COSTES[i][S[i]]

    #Estimacion
    for i in range( len(S), len(COSTES) ):
        VALOR += min( [ COSTES[j][i] for j in range(len(S), len(COSTES)) ])
    return VALOR

def CS(S,COSTES):
    VALOR = 0
    #Valores establecidos
    for i in range(len(S)):
        VALOR += COSTES[i][S[i]]

    #Estimacion
    for i in range( len(S), len(COSTES) ):
        VALOR += max( [ COSTES[j][i] for j in range(len(S), len(COSTES)) ])
    return VALOR

print(CI((0,1),COSTES))
print(CS((0,1),COSTES))
print(len(COSTES))
```

```
68
74
4
```

```
#Genera tantos hijos como como posibilidades haya para la siguiente elemento de la tupla
#(0,) -> (0,1), (0,2), (0,3)
def crear_hijos(NODO, N):
    HIJOS = []
    for i in range(N ):
        if i not in NODO:
            HIJOS.append({'s':NODO +(i,) })
    return HIJOS
```

```
crear_hijos((0,) , 4)
```

```
[{'s': (0, 1)}, {'s': (0, 2)}, {'s': (0, 3)}]
```

```
def ramificacion_y_poda(COSTES):
#Construcción iterativa de soluciones(arbol). En cada etapa asignamos un agente(ramas).
#Nodos del grafo { s:(1,2),CI:3,CS:5 }
#print(COSTES)
DIMENSION = len(COSTES)
MEJOR_SOLUCION=tuple( i for i in range(len(COSTES)) )
CotaSup = valor(MEJOR_SOLUCION,COSTES)
#print("Cota Superior:", CotaSup)

NODOS=[]
NODOS.append({'s':(), 'ci':CI((),COSTES) } )

iteracion = 0

while( len(NODOS) > 0):
    iteracion +=1

    nodo_prometedor = [ min(NODOS, key=lambda x:x['ci']) ][0]['s']
    #print("Nodo prometedor:", nodo_prometedor)

    #Ramificacion
    #Se generan los hijos
    HIJOS = [ {'s':x['s'], 'ci':CI(x['s'], COSTES) } for x in crear_hijos(nodo_prometedor, DIMENSION) ]

    #Revisamos la cota superior y nos quedamos con la mejor solucion si llegamos a una solucion final
    NODO_FINAL = [x for x in HIJOS if len(x['s']) == DIMENSION ]
    if len(NODO_FINAL) >0:
        #print("\n*****Soluciones:", [x for x in HIJOS if len(x['s']) == DIMENSION ] )
        if NODO_FINAL[0]['ci'] < CotaSup:
            CotaSup = NODO_FINAL[0]['ci']
            MEJOR_SOLUCION = NODO_FINAL

    #Poda
    HIJOS = [x for x in HIJOS if x['ci'] < CotaSup ]

    #Añadimos los hijos
    NODOS.extend(HIJOS)

    #Eliminamos el nodo ramificado
    NODOS = [ x for x in NODOS if x['s'] != nodo_prometedor ]

    print("La solucion final es:",MEJOR_SOLUCION , " en " , iteracion , " iteraciones" , " para dimension: " ,DIMENSION )

ramificacion_y_poda(COSTES)
```

La solucion final es: [{'s': (1, 2, 0, 3), 'ci': 64}] en 10 iteraciones para dimension: 4

▼ Descenso del gradiente

```
import math                #Funciones matematicas
import matplotlib.pyplot as plt #Generacion de gráficos (otra opcion seaborn)
import numpy as np         #Tratamiento matriz N-dimensionales y otras (fundamental!)
import scipy as sc

import random
```

Vamos a buscar el minimo de la funcion paraboloide por le metodo del descenso del gradiente :

$$f(x) = x^2 + y^2$$

Obviamente se encuentra en (x,y)=(0,0) pero probaremos como llegamos a él a través del descenso del gradiante.

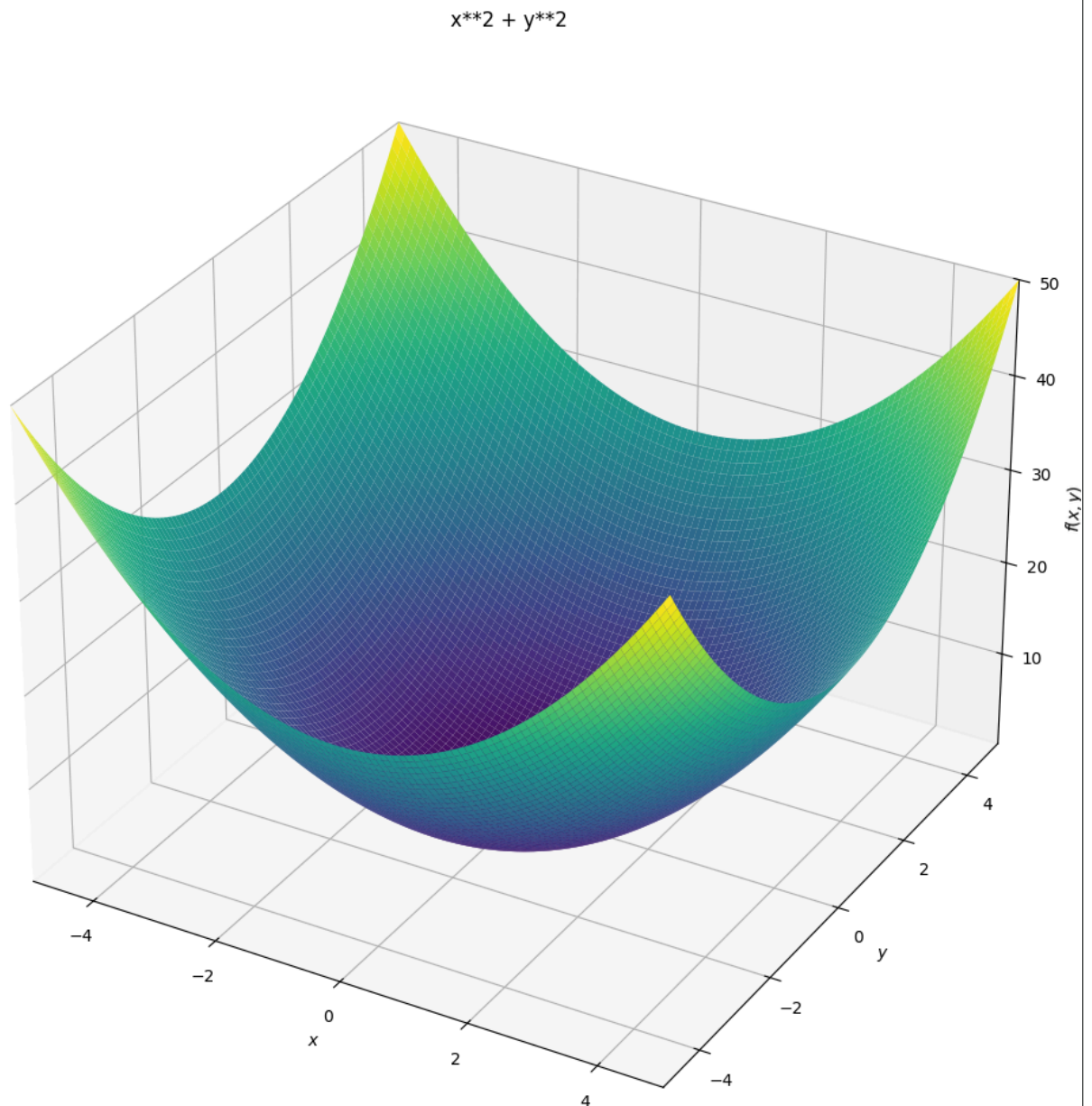
```
#Definimos la funcion(f) y el gradiente(df)
#Paraboloide
f = lambda X:      X[0]**2 + X[1]**2      #Funcion
df = lambda X: [2*X[0] , 2*X[1]]          #Gradiente

df([1,2])

[2, 4]
```

```
from sympy import symbols
from sympy.plotting import plot
from sympy.plotting import plot3d
```

```
x,y = symbols('x y')
plot3d(x**2 + y**2,
      (x,-5,5),(y,-5,5),
      title='x**2 + y**2',
      size=(10,10))
```



<sympy.plotting.backends.matplotlibbackend.matplotlib.MatplotlibBackend at 0x7fdea731c290>

```
#Prepara los datos para dibujar mapa de niveles de Z
resolucion = 100
rango=2.5

X=np.linspace(-rango,rango,resolucion)
Y=np.linspace(-rango,rango,resolucion)
Z=np.zeros((resolucion,resolucion))
for ix,x in enumerate(X):
    for iy,y in enumerate(Y):
        Z[iy,ix] = f([x,y])

#Pinta el mapa de niveles de Z
plt.contourf(X,Y,Z,resolucion)
plt.colorbar()

#Generamos un punto aleatorio inicial y pintamos de blanco
P=[random.uniform(-2,2 ),random.uniform(-2,2 ) ]
plt.plot(P[0],P[1],"o",c="white")

#Tasa de aprendizaje. Fija. Sería más efectivo reducirlo a medida que nos acercamos.
TA=.1

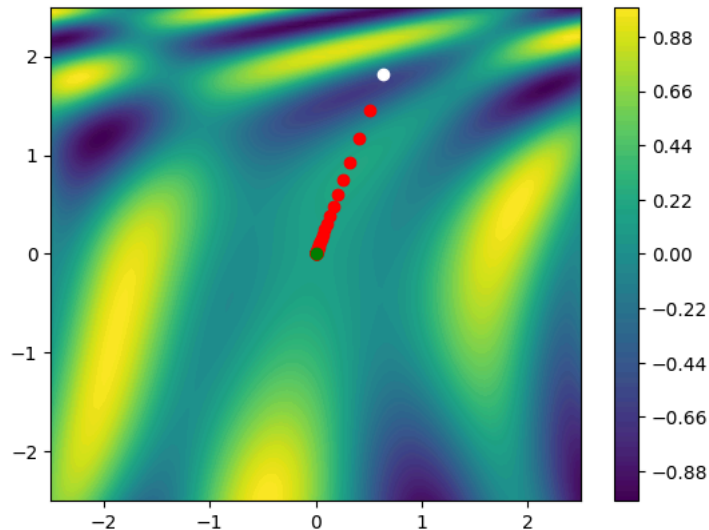
#Iteraciones:500
```

```

for _ in range(50):
    grad = df(P)
    #print(P,grad)
    P[0],P[1] = P[0] - TA*grad[0] , P[1] - TA*grad[1]
    plt.plot(P[0],P[1],"o",c="red")

#Dibujamos el punto final y pintamos de verde
plt.plot(P[0],P[1],"o",c="green")
plt.show()
print("Solucion:" , P , f(P))

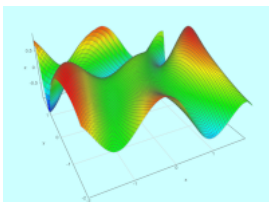
```



Solucion: [9.077728655348086e-06, 2.596287268986727e-05] 0.14112000818160675

¿Te atreves a optimizar la función?:

$$f(x) = \sin(1/2 * x^2 - 1/4 * y^2 + 3) * \cos(2 * x + 1 - e^y)$$



```

#Definimos la funcion
f= lambda X: math.sin(1/2 * X[0]**2 - 1/4 * X[1]**2 + 3) *math.cos(2*X[0] + 1 - math.exp(X[1])) #Funcion

# df = lambda X: [X[0]*math.cos(2*X[0]-math.exp(X[1]))+1]*math.cos(X[0]**2 /2 -X[1]**2 /4 + 3) - 2*math.sin(2*X[0] - math.e

def df(PUNTO):
    h=0.01
    PUNTO = np.asarray(PUNTO, dtype=float)
    T = PUNTO.copy()
    grad = np.zeros(2, dtype=float)

    for it, _ in enumerate(PUNTO):
        T = PUNTO.copy()          # (clave) partir siempre del punto original
        T[it] = T[it] + h         # X + h*e_it
        grad[it] = (f(T) - f(PUNTO)) / h

    return grad

```

```

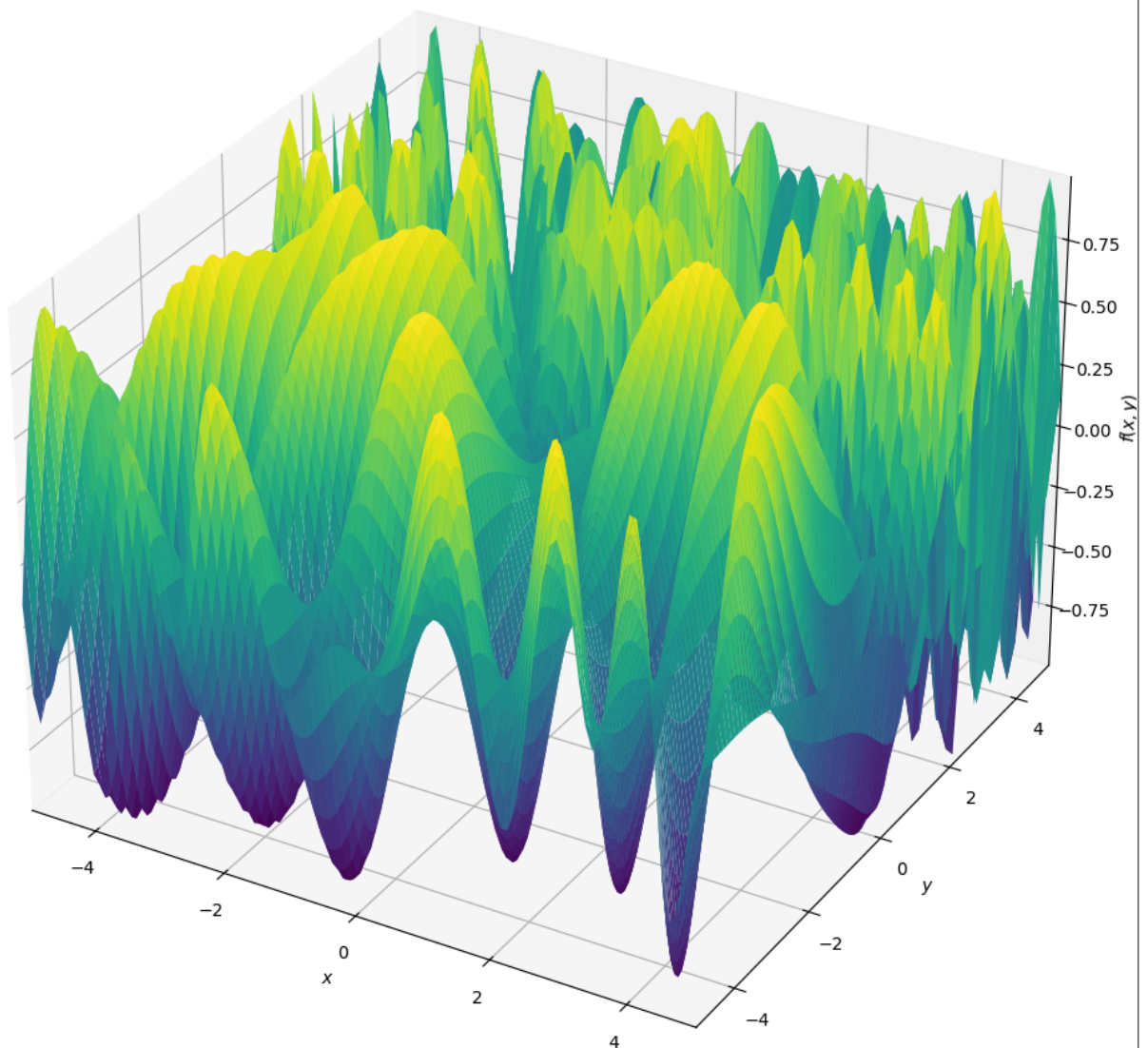
from sympy import symbols, sin, cos, exp
from sympy.plotting import plot3d

x, y = symbols('x y')

plot3d(
    sin(0.5*x**2 - 0.25*y**2 + 3) * cos(2*x + 1 - exp(y)),
    (x, -5, 5), (y, -5, 5),
    title='sin(1/2 * x**2 - 1/4 * y**2 + 3) * cos(2*x + 1 - exp(y))',
    size=(10, 10)
)

```

$$\sin(1/2 * x^2 - 1/4 * y^2 + 3) * \cos(2*x + 1 - \exp(y))$$



<sympy.plotting.backends.matplotlibbackend.matplotlib.MatplotlibBackend at 0x7fdeb92fd550>

```
#Prepara los datos para dibujar mapa de niveles de Z
resolucion = 100
rango=2.5

X=np.linspace(-rango,rango,resolucion)
Y=np.linspace(-rango,rango,resolucion)
Z=np.zeros((resolucion,resolucion))
for ix,x in enumerate(X):
    for iy,y in enumerate(Y):
        Z[iy,ix] = f([x,y])

#Pinta el mapa de niveles de Z
plt.contourf(X,Y,Z,resolucion)
plt.colorbar()

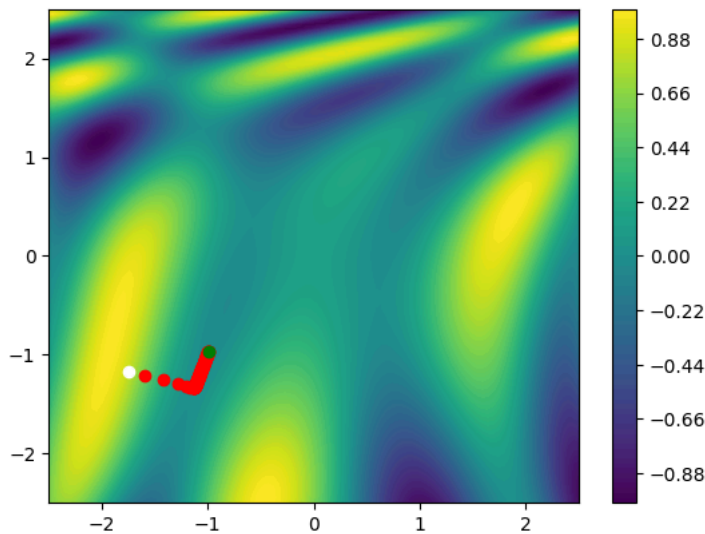
#Generamos un punto aleatorio inicial y pintamos de blanco
P=[random.uniform(-2,2 ),random.uniform(-2,2 ) ]
plt.plot(P[0],P[1], "o",c="white")

#Tasa de aprendizaje. Fija. Sería más efectivo reducirlo a medida que nos acercamos.
TA=.1

#Iteraciones:500
for _ in range(100):
    grad = df(P)
    #print(P,grad)
    P[0],P[1] = P[0] - TA*grad[0] , P[1] - TA*grad[1]
    plt.plot(P[0],P[1], "o",c="red")
```



```
#Dibujamos el punto final y pintamos de verde
plt.plot(P[0],P[1],"o",c="green")
plt.show()
print("Solucion:" , P , f(P))
```



Solucion: [np.float64(-0.993879165316108), np.float64(-0.9644353060199231)] -0.02395343306076632

✓ Problema de Autobuses por Fuerza Bruta

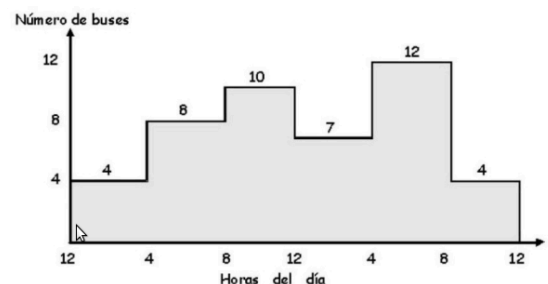
Desarrollo e implementación de algoritmos. Ejemplo

Una pequeña ciudad estudia introducir un sistema de transporte urbano de autobuses. Nos encargan estudiar el número mínimo de autobuses para cubrir la demanda. Se ha realizado un estudio para estimar el número mínimo de autobuses por franja horaria. Lógicamente este número varia dependiendo de la hora del día. Se observa que es posible dividir la franja horaria de 24h en tramos de 4 horas en los queda determinado el número de autobuses que se necesitan. Debido a la normativa cada autobús debe circular 8 horas como máximo y seguidas en cada jornada de 24h.

viu Universidad
Internacional
de Valencia



8 horas x día



Pg.: 12

```
#####
# Ximo Sanz Tornero.
#####
from itertools import product

# Paso 1: Inicializamos los datos
# Demanda mínima de autobuses por tramo
demanda = [4, 8, 10, 7, 12, 4] # d[0], d[1], ..., d[5]
tramos = len(demanda) # Número de tramos (6 en este caso)

#Posible Solucion
x = [4,5,6,7,8,9] # Partimos de una posible solución inicial

for t in range(tramos):
    # Calculamos el número actual de autobuses que están cubriendo el tramo t
    cobertura_actual = x[t] + x[t - 1] # Autobuses en t y t-1 (cíclico)

    # Si la cobertura actual es menor que la demanda, añadimos autobuses en t
    if cobertura_actual < demanda[t]:
```



```

if cobertura_actual < demanda[t]:
    # Añadimos los autobuses necesarios en el tramo t
    x[t] += demanda[t] - cobertura_actual

#Función objetivo
f_objetivo = sum(x)

# Se crean funciones tanto para verificar posibles soluciones como para obtener el valor de la función objetivo de la solución
# Función objetivo: suma de todos los x[i]
def funcion_objetivo(solucion):
    return sum(solucion)

# Verificación de restricciones
def verifica_restricciones(solucion):
    for t in range(tramos): # 0(6)
        # Se calcula el número actual de autobuses que están cubriendo el tramo t
        cobertura_actual = solucion[t] + solucion[t - 1] # Cíclico # 0(6)
        if cobertura_actual < demanda[t]: # 0(6)
            return False
    return True

# Se realiza el algoritmo por fuerza bruta con todas las posibles combinaciones de valores que tenemos en función de la variable
# Generación del espacio de soluciones
# Para ver mejor el funcionamiento, ejecutar: for solucion in product(*(range(d + 1) for d in [1,2,3])): print(solucion)
espacio_soluciones = product(*(range(d + 1) for d in demanda))

# Búsqueda de la mejor solución
mejor_valor_objetivo = float('inf')
mejores_soluciones = []

for solucion in espacio_soluciones: # Tiene complejidad exponencial en el número de tramos
    if verifica_restricciones(solucion):
        valor_objetivo = funcion_objetivo(solucion)
        if valor_objetivo < mejor_valor_objetivo:
            mejor_valor_objetivo = valor_objetivo
            mejores_soluciones = [solucion]
        elif valor_objetivo == mejor_valor_objetivo:
            mejores_soluciones.append(solucion) # Agrega soluciones con el mismo valor

# Resultado
print("Mejores soluciones:")
for solucion in mejores_soluciones:
    print(solucion)
print("Valor mínimo de la función objetivo:", mejor_valor_objetivo)

# El algoritmo por fuerza bruta deja de ser una opción a partir de dimensiones moderadas (≈7-8 tramos), ya que el número de

```

```

(1, 8, 2, 6, 6, 3)
(1, 8, 2, 7, 5, 3)
(2, 6, 4, 3, 9, 2)
(2, 6, 4, 4, 8, 2)
(2, 6, 4, 5, 7, 2)
(2, 6, 4, 6, 6, 2)
(2, 6, 4, 7, 5, 2)
(2, 7, 3, 4, 8, 2)
(2, 7, 3, 5, 7, 2)
(2, 7, 3, 6, 6, 2)
(2, 7, 3, 7, 5, 2)
(2, 8, 2, 5, 7, 2)
(2, 8, 2, 6, 6, 2)
(2, 8, 2, 7, 5, 2)
(3, 5, 5, 2, 10, 1)
(3, 5, 5, 3, 9, 1)
(3, 5, 5, 4, 8, 1)
(3, 5, 5, 5, 7, 1)
(3, 5, 5, 6, 6, 1)
(3, 5, 5, 7, 5, 1)
(3, 6, 4, 3, 9, 1)
(3, 6, 4, 4, 8, 1)
(3, 6, 4, 5, 7, 1)
(3, 6, 4, 6, 6, 1)
(3, 6, 4, 7, 5, 1)

```

```
(4, 4, 0, 5, 9, 0)
(4, 4, 6, 4, 8, 0)
(4, 4, 6, 5, 7, 0)
(4, 4, 6, 6, 6, 0)
(4, 4, 6, 7, 5, 0)
(4, 5, 5, 2, 10, 0)
(4, 5, 5, 3, 9, 0)
(4, 5, 5, 4, 8, 0)
(4, 5, 5, 5, 7, 0)
(4, 5, 5, 6, 6, 0)
(4, 5, 5, 7, 5, 0)
(4, 6, 4, 3, 9, 0)
(4, 6, 4, 4, 8, 0)
(4, 6, 4, 5, 7, 0)
(4, 6, 4, 6, 6, 0)
(4, 6, 4, 7, 5, 0)
(4, 7, 3, 4, 8, 0)
(4, 7, 3, 5, 7, 0)
(4, 7, 3, 6, 6, 0)
(4, 7, 3, 7, 5, 0)
(4, 8, 2, 5, 7, 0)
(4, 8, 2, 6, 6, 0)
(4, 8, 2, 7, 5, 0)
```

Valor mínimo de la función objetivo: 26

```
for solucion in product(*(range(d + 1) for d in [1,2,3])): print(solucion)
```

```
(0, 0, 0)
(0, 0, 1)
(0, 0, 2)
(0, 0, 3)
(0, 1, 0)
(0, 1, 1)
(0, 1, 2)
(0, 1, 3)
(0, 2, 0)
(0, 2, 1)
(0, 2, 2)
(0, 2, 3)
(1, 0, 0)
(1, 0, 1)
(1, 0, 2)
(1, 0, 3)
(1, 1, 0)
(1, 1, 1)
(1, 1, 2)
(1, 1, 3)
(1, 2, 0)
(1, 2, 1)
(1, 2, 2)
(1, 2, 3)
```

Empieza a programar o a [crear código](#) con IA.